
APPENDIX A: A BRIEF OVERVIEW OF GOF DESIGN PATTERNS

We all have unique thought processes. So, in the early days of software development, engineers faced a common problem. There was no standard to instruct them on how to design their applications. Each team followed its own style, and when a new member joined an existing team, understanding the architecture was a gigantic task. Senior or experienced members of the team had to explain to the new joiner about the advantages of the existing architecture and why alternative designs were not considered.

The experienced developer also knew how to reduce future efforts by simply reusing the concepts already in place. Design patterns address this issue and provide a common platform for all developers. You can think of them as the recorded experience of experts in the field. These patterns were intended to be applied in object-oriented designs with the intention of reuse.

In 1994, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides published the book *Design Patterns: Elements of Reusable Object-Oriented Software* (Addison-Wesley, 1994). In this book, they introduced the concept of design patterns in software development. *These authors became known as the Gang of Four. I refer to them as the GoF throughout this book.* The GoF described 23 patterns that were developed by the common experiences of software developers over a while. Nowadays, when new members join a development team, they are expected to know about the design patterns and then they learn about the existing architecture. This approach allows them to actively participate in the development process within a short period.

The first concept of a real-life design pattern came from the building architect Christopher Alexander. During his lifetime, he discovered that many of the problems he

faced were similar in nature. So, he tried to address those issues with similar types of solutions.

Each pattern describes a problem, which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

Christopher Alexander

The software engineering community started believing that although these patterns were described for buildings and towns, the same concepts could be applied to patterns in object-oriented design. So, they substituted the original concepts of walls and doors with objects and interfaces. The common thing in both fields is that, at their cores, patterns are solutions to common problems.

Lastly, it is important to note that the GoF discussed the original concepts of design patterns in the context of C++. But Sun Microsystem released its first public implementation of Java 1.0 in 1995 and then it went through various changes. So, in 1995, Java was totally new to the programming world. But it grew rapidly and secured its rank in the world's top programming languages within a short period of time. In today's market, it is always in high demand. On the other hand, the concepts of design patterns are universal. So, when you exercise these fundamental concepts of design patterns with Java, you will be a better programmer and you'll remake yourself in the programming community. Here are some important points to remember:

- A design pattern describes a general reusable solution to software design problems. While developing software, you may encounter these problems frequently. The basic idea is that you can solve similar kinds of problems with similar kinds of solutions. And these solutions have been tested over a long period.
- These patterns provide you with a template of how to solve a problem and can be used in many different situations. At the same time, they help you to get the best possible design much faster.
- These patterns are descriptions of how to create objects and classes and then customize them to solve a general design problem in a particular context.

The GoF discussed 23 design patterns. Each of these patterns focuses on a particular object-oriented design. Each pattern can also describe the con-

sequences and tradeoffs of use. The GoF categorized these 23 patterns based on their purposes, as shown here:

A. Creational patterns: These patterns abstract the instantiation process. You make the systems independent from how their objects are composed, created, and represented. In these patterns, you should have a basic concern: “Where should I place the ‘new’ keyword in my application?” This decision can determine the degree of coupling of your classes. The following five patterns belong to this category:

- Singleton pattern
- Prototype pattern
- Factory Method pattern
- Builder pattern
- Abstract Factory pattern

B. Structural patterns: Here you focus on how classes and objects can be composed to form a relatively large structure. They generally use inheritance or composition to group different interfaces or implementations. Your choice of composition over inheritance (and vice versa) can affect the flexibility of your software. The following seven patterns fall into this category:

- Proxy pattern
- Flyweight pattern
- Composite pattern
- Bridge pattern
- Facade pattern
- Decorator pattern
- Adapter pattern

C. Behavioral patterns: Here you concentrate on algorithms and the assignment of responsibilities among objects. You also need to focus on the communication between them and how the objects are interconnected. The following eleven patterns fall into this category:

- Observer pattern
- Strategy pattern
- Template Method pattern
- Command pattern
- Iterator pattern
- Memento pattern

- State pattern
- Mediator pattern
- Chain of Responsibility pattern
- Visitor pattern
- Interpreter pattern

The GoF made another classification based on scope. It examines whether the pattern primarily focuses on the classes or its objects. Class patterns deal with classes and subclasses. They use inheritance mechanisms, so they are static and fixed at compile time. Object patterns deal with objects that can change at run time. So, object patterns are dynamic.

For a quick reference, you can refer to Table [A-1](#), which was introduced by the GoF.

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	1. Factory Method	1.1. Adapter(class)	1. Interpreter 2. Template Method
	Object	2. Singleton 3. Prototype 4. Builder 5. Abstract Factory	1.2. Adapter(object) 2. Proxy 3. Flyweight 4. Composite 5. Bridge 6. Facade 7. Decorator	3. Observer 4. Strategy 5. Command 6. Iterator 7. Memento 8. State 9. Mediator 10. Visitor 11. Chain of Responsibility

Table A-1 Classification of the GoF Design Patterns

Q&A SESSION

A1.1 What are the differences between class patterns and object patterns?

In general, class patterns focus on static relationships, but object patterns can focus on dynamic relationships. As the names suggest, class patterns focus on classes and their subclasses, and object patterns focus on the object's relationships.

Table [A-2](#) shows the summarized content that was discussed in the GoF's famous book.

Table A-2 Needs Caption

	Class patterns	Object patterns
Creational	Can defer object creation to subclasses	Can defer object creation to another object
Structural	They focus on the composition of classes (primarily using the concept of inheritance).	They focus on the different ways to assemble objects.
Behavioral	Describes the algorithms and execution flows. They also use the inheritance mechanism.	Describes how different objects can work together and complete a task.

A1.2 Can I combine two or more patterns in an application?

Yes. In real-world scenarios, this type of activity is common.

A1.3 Are these patterns dependent on a particular programming language?

Programming languages can play an important role. But the basic ideas are the same. Patterns are just like templates and they will give you some idea in advance of how you can solve a particular problem. In this book, I primarily focus on object-oriented programming with the concept of reuse. But instead of a object-oriented programming language, suppose you have chosen some other language like C. In that case, you may need to think about the core object-oriented principles such as inheritance, polymorphism, encapsulation, abstraction, and so on, and how to implement them. So, the choice of a particular language is always important because it may have some specialized features that can make your programming life easier.

A1.4 Should I consider common data structures like arrays and linked lists as different design patterns?

The GoF excludes these, saying that *they are not complex, domain-specific designs for an entire application or subsystem*. They can be encoded in classes and reused as-is. So, they are not your concern in this book.

A1.5 If no particular pattern is 100% suitable for my problem, how should I proceed?

An infinite number of problems cannot be solved with a finite number of patterns for sure. But if you know these common patterns and their trade-offs, you can pick a close match. Lastly, no one prevents you from using a pattern that serves your need. But you have to tackle the risk and you need to think about your return on investment.

A1.6 Any general advice before I jump into the topics?

I prefer to follow in the footsteps of my seniors and teachers who are experts in this field. And here are some general suggestions from them:

- Program to a supertype (abstract class/interface), not an implementation.
- Prefer composition over inheritance in most cases.
- Try to make a loosely coupled system.
- Segregate code that is likely to vary from the rest of your code.
- Encapsulate what varies.

A1.7 How can I use this book effectively?

This book focuses on commonly used design patterns. Most likely, you will face them very often in your everyday life. But the world is always changing, so known patterns keep evolving and many new patterns are created. To understand the necessity of a new pattern, you may also need to understand why an old/existing pattern is not enough to fulfill the requirement. You may consider this book as an attempt to make a solid foundation with design patterns, so that you can code smoothly in your professional life and you can adapt to upcoming changes easily.

APPENDIX B: THE ROAD AHEAD

Congratulations! You have reached the end of the journey. Anyone can start a journey, but only a few can complete it with care. So, you are among the minority who possess the extraordinary capability to cover the distance successfully. I believe that you have enjoyed your learning experience, which can help you to learn and experiment further in this category. I said earlier that if you repeatedly think about the discussions, examples, implementations, and the Q&A Sessions in the book, you will have more clarity about them, you will feel more confident about them, and you will remake yourself in the programming world.

Truly, a detailed discussion of any particular design pattern in depth would need many more pages and then the size of the book would be too gigantic to digest. In addition, although you have seen the GoF design patterns and a few more patterns, you can surely guess that there are many more pattern catalogs including domain-specific patterns such as enterprise patterns, concurrent system patterns, etc. So, what's next? You should not forget the basic principle that *learning is a continuous process*. So, this book was an attempt to encourage you to learn the fundamental patterns in depth so that you can enjoy your future learning in this field.

However, learning and thinking by yourself will not be enough. You can participate in open forums and join discussion groups to get more clarity on this subject. This is why searching in Google, Stack Overflow, Quora, or similar places can help you a lot. The only caution is that you must be careful when processing that information. How will the clarity come? Keep analyzing and see whether you are satisfied with the answer. This process will not only help you; it will help others also.

A PERSONAL APPEAL TO YOU

Over the years, I have seen a general trend for my books. When you like the book, you send me messages, write nice emails, and motivate me with your kind words and suggestions. But many of them do not reach review platforms like Amazon and others. But when the opposite happens, I see all the criticisms on those pages.

Please know that these criticisms help me to write better. But it's also helpful to know what you liked about a book. These constructive sug-

gestions help me to keep that information in an updated edition of the book.

So, I have a request for you: You can always point out the improvement areas of this work but at the same time, please let me know about the areas you liked. In general, it is always easy to criticize, but an artistic view and open mind are required to discover the true efforts that are associated with any kind of work. Thank you and happy coding!

APPENDIX C: RECOMMENDED READING

This appendix lists some useful resources. I have learned from them.

I believe that you'll benefit from these great books (or the updated editions):

- *Design Patterns: Elements of Reusable Object-Oriented Software* by Erich Gamma et al. (Addison-Wesley, 1995).
- *Head First Design Patterns* by Eric Freeman and Elisabeth Robson (O'Reilly, 2004).
- *Design Patterns for Dummies* by Steve Holzner (Wiley Publishing, Inc, 2006).
- *Java Design Pattern Essentials* by Tony Bevis (Ability First Limited, 2012).

In today's world, online platforms are very powerful. In Appendix B, I told you that you can also learn from online platforms such as Google, Stack Overflow, Quora, YouTube, Udemy, etc. You can use the power of these online platforms to learn not only design patterns but almost anything you want. But you must validate that information before you process it. Here I include some helpful online resources/websites for your immediate reference:

- http://sourcemaking.com/design_patterns
- https://en.wikipedia.org/wiki/Software_design_pattern
- www.codeproject.com/
- <https://dzone.com/>
- www.tutorialspoint.com/
- www.udemy.com/courses/search/?src=ukw&q=design+patterns
- https://en.wikipedia.org/wiki/Design_pattern
- <http://wiki.c2.com/?AntiPatternsCatalog>

- <http://hillside.net>
-

Index

A

Abstract Factory pattern

analysis

class diagram

client code variations

demonstration

implementation

output

package explorer view

Q&A

Abstract Window Toolkit (AWT)

Adapter pattern

analysis

demonstration

implementation

analysis

class diagram

demonstration

output

package explorer view

output

Adapters and decorators

AlternationExpression, RepetitionExpression/SequenceExpression

animal.displayBehavior() analysis

AnimalFactory class

Anti-patterns

code

definition

examples

real-world application development

refactoring

symptoms

types

B

Bridge pattern

adapter design

advantages

connection

Driver

GUI frameworks

implementation

class

class diagram

design

flexibility

implementors

interface

modified program

output

package

package explorer view

structure

interface

real-life example

Builder pattern

computer world example

definition

immutable object

implementation

advantages

class diagram

data structure

demonstration

interface

interfaces

output

package explorer view

modified implementation

analysis

characteristics

demonstration

output

parameters

C

Chain of Responsibility pattern

advantages

catch block

disadvantages

e-mail/faxes

implementation

class diagram

code

fax
output
package explorer view
program
mobile company organization
try/catch/finally blocks
checkSeniority() method
Class adapters
Client-server architecture
clone() method
Command pattern
advantages
client code
command objects
concept
disadvantages
implementation
class diagram
demonstration
output
package explorer view
invoker
modified implementation
class diagram
demonstration
output
undo
Composite pattern
analysis
class diagram
concept
demonstration
implementation
output
package explorer view
Computer Science and Engineering (CSE)
conveyThanks() method
courseOnAdditionalPaper()
courseOnSpecialPaper() method
createAndDisplayAnimal() method
createAnimal() method
Creator class

D

Database programming
Decorator pattern
advantages
BufferedOutputStream
computer world example
customers
disadvantages
hierarchical inheritance
implementation
demonstration
object composition
output
subclassing
object
real-life example
decrementNumber() method
Dependency Inversion Principle (DIP)
analysis
better program
demonstration
initial program
output
Design patterns
anti-patterns
definition
developers
GoFs
writing new pattern
displayBehavior() method
displayCourseStructure() method
displayEmpDetail() method
displayProductPrice(...) method
doSomeWork() method
dummyMethod()

E

EmailErrorHandler and FaxErrorHandler
evaluateDistinction() method
evaluateDistinction(...) method
execute() method

F

Facade pattern
advantages
code

computer world example

definition

facades

implementations

analysis

assumptions

class diagram

constructor

demonstration

output

multiple methods

real world example

Factory Method pattern

class diagram

demonstration

implementation

inheritance hierarchies

modified implementation

analysis

demonstration

output

output

package explorer view

Q&A

Factory of factories

festiveSeasonDiscount() method

Flyweight pattern

analysis

class diagram

demonstration

implementation

output

package explorer view

G

generateEmpId() method

getCaptain() method

getConnection()

getDiscount() method

getDoubleDiscountWithThanks() method

getter-setter methods

Graphical user interface (GUI)

H

Handle/Body pattern

hook method

I

incrementNumber() method

Infix notation

Inheritance and polymorphism

instruct() method

Interface Segregation Principle (LSP)

analysis

better program

demonstration

initial program

output

interpret() method

interpret(...) method

Interpreter pattern

characteristics

expressions

grammar

implementation

advantages

disadvantages

class diagram

context class

demonstration

demonstration

expressions

grammar

OrExpression

output

package explorer view

MSIL

output

structure

iterator() method

Iterator pattern

collections

demonstration

implementation

class diagram

comments

demonstration

output

package explorer view

output

participants

usage

Iterators

J

Java Design Patterns

Java Virtual Machine

jdp3e.facade package

K

Keep It Simple, Stupid (KISS) principle

L

Liskov Substitution Principle (LSP)

analysis

better program

demonstration

initial program

output

M

main() method

Mediator pattern

additional implementation

analysis

demonstration

output

advantages

concept

disadvantages

implementation

analysis

class diagram

demonstration

output

package explorer view

Mediator Pattern

Memento pattern

additional implementation

analysis

class diagram

demonstration

output

package explorer view

advantages

concept

- disadvantages
- implementation
- analysis
- class diagram
- demonstration
- output
- package explorer view
- Method chaining
- Microsoft Intermediate Language (MSIL) code
- MobileDeviceView
- Model-specific parts
- Model–view–controller (MVC) pattern
- Model-View-Presenter (MVP)
- Model-View-ViewModel (MVVM)
- N**
- newInstance() method
- NewPayments() method
- non-abstract methods
- Null Object pattern
- advantages
- analysis
- applications
- cases
- client-server architecture
- components
- controller
- DBA
- disadvantages
- faulty program
- implementation
- analysis
- class diagram
- code
- demonstration
- NullVehicle object
- output
- output
- package explorer view
- parts
- View interface
- invoking operation
- Java’s Swing components
- null collaborations check

output
Package Explorer view
proxies
structure
unwanted input
variations
washing machine

O

Object adapters
Observable object's notify Observers() method
Observer
additional methods
advantages
class diagram
code
definition
disadvantages
implementation
instances
interface
output
package explorer view
subject
UI based example
Observer pattern
Open/Closed Principle (OCP)
analysis
better program
demonstration
initial program
output
openStream() method

P, Q

Palo Alto Research Center (PARC)
Pattern Languages of Program Design (PLoP)
PetAnimalFactory
Portland Patterns Repository
previousPaymentInfo() method
PriceType interface
printDocument() method
Prototype pattern
advantages
cloning method

computer world example

design pattern

examples

implementation

class diagram

demonstration

example

output

package explorer review

modified implementation

analysis

analysis

CarMaker

class diagram

output

Singleton pattern

shallow copy vs. deep copy

analysis

definition

demonstration

implementation

output

shallow copy vs. deep copy

analysis

references

Proxy pattern

class constructor

computer world example

definition

implementation

class diagram

demonstration

output

package explorer view

output

real life example

remote proxy

types

virtual proxy

Publisher-Subscriber (Pub-Sub) model

R

Refactoring

register(...), unregister(...), and notifyRegisteredUsers()

- removeAll() method
- Request-handling mechanism
- Rich Text Format (RTF)
- S**
- saveEmployeeId() method
- sayThanks() method
- sendFax() method
- SeniorityChecker class
- setChanged method
- setter method
- showPriceDetail() method
- Simple Factory pattern
 - analysis
 - class diagram
 - demonstration
 - implementation
 - output
 - package explorer view
- Q&A
- software applications
- usage
- Single responsibility principle (SRP)
 - analysis
 - better program
 - demonstration
 - initial program
 - output
- Singleton pattern
 - analysis
- Bill Pugh solution
 - computer life example
 - definition
 - enum
 - implementation
 - analysis
 - characteristics
 - class diagram
 - demonstration
 - output
 - initialization
 - multiple object creation
 - real life example
 - test code

- size() method
- Software applications
- State pattern
- advantages
- concept
- implementation
- analysis
- class diagram
- demonstration
- output
- package explorer view
- Thread class
- Strategy pattern
- advantages
- Boat class
- concept
- context class
- disadvantages
- implementation
- class diagram
- demonstration
- output
- package explorer view
- Synchronization

T

- Template Method pattern
- advantages
- definition
- disadvantages
- implementation
- class diagram
- output
- program
- source code
- structure
- output
- parent class methods
- toLowerCase() method
- toString() method
- travel() method
- try/catch/finally block

U

- Uncle Bob

V

validateExpression(...) method

visitNumber(...) method

Visitor pattern

analysis

appraisal process

demonstration

drawbacks

implementation

analysis

class diagram

demonstration

IntegerProcessor class

NumberProcessor hierarchy

output

package explorer view

setter method

Visitor hierarchy

interface

object structure

output

public APIS

structure

use cases

W

WildAnimalFactory

Windows Presentation Foundation (WPF)

X, Y, Z

XMLReaderFactory

visitXyz method

Previous chapter

< 30. FAQ